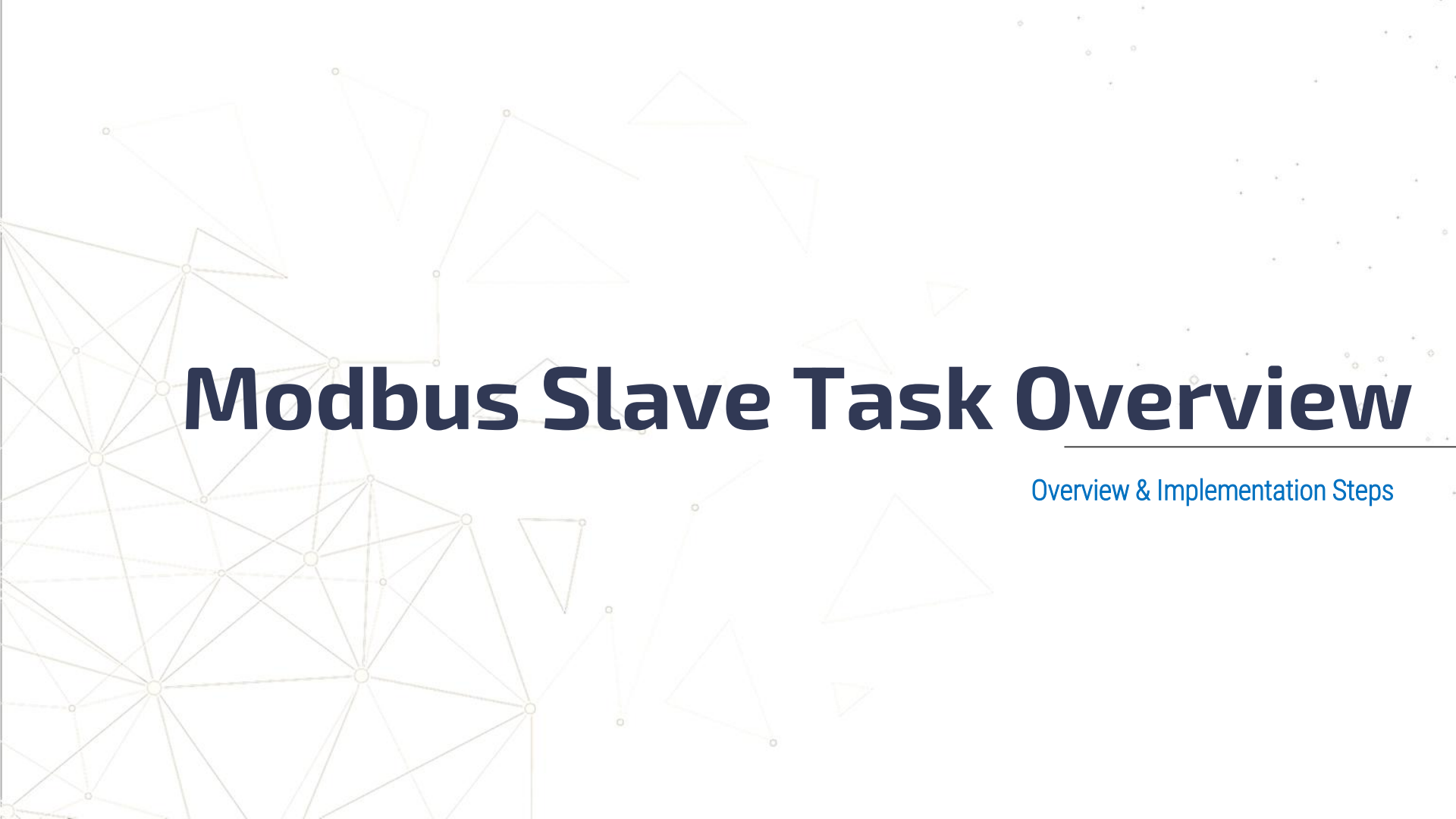




Modbus Slave Task Overview

Overview & Implementation Steps

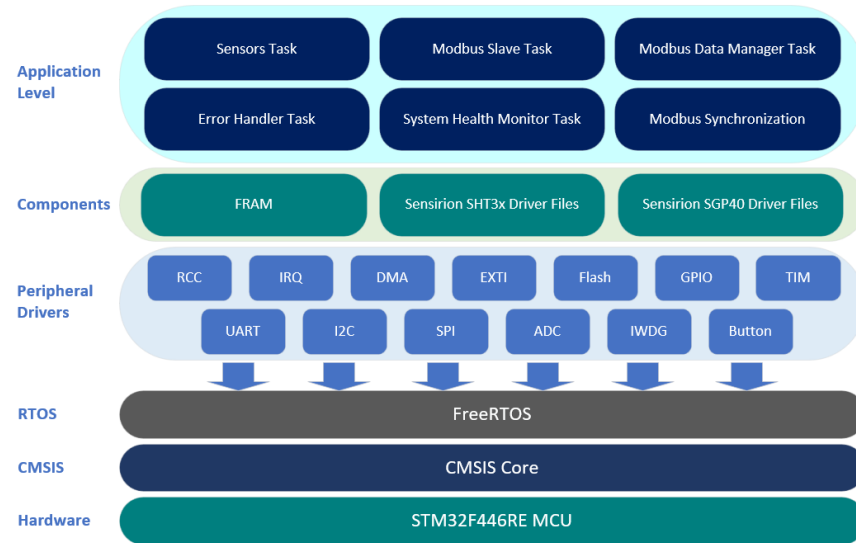
Modbus Slave Task Overview

- Modbus Slave Task Overview
- Section Implementation Steps

Modbus Slave Task Overview Part I

General

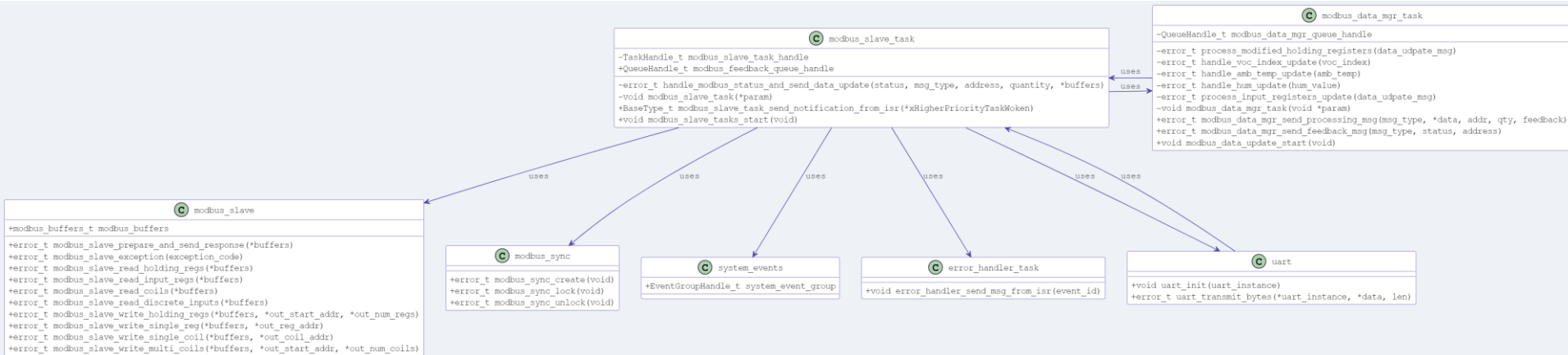
- **Purpose & Functionality:** Handles Modbus RTU requests, facilitates reading/writing registers, and manages coils and discrete inputs.
- **Project Significance:**
 - **Core Communication Interface:** Responsible for processing requests and preparing responses to ensure seamless data exchange.
 - **UART Integration:** The Modbus task is triggered when the UART detects the end of transmission using the IDLE line detection interrupt, signaling that a frame is ready for processing.



Modbus Slave Task Overview Part II

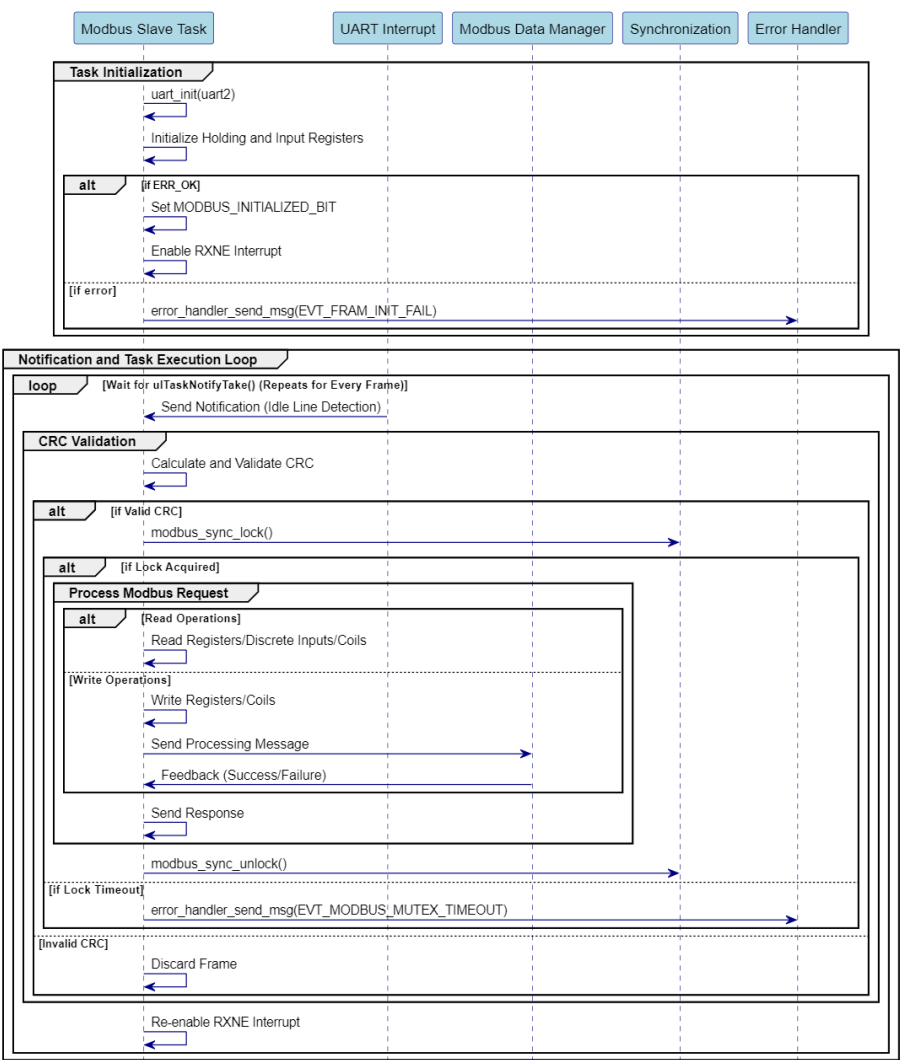
FreeRTOS Task Activities

- The Modbus Slave Task waits for UART interrupt task notifications signaling the end of a Modbus RTU frame.
- Validates the CRC, processes the request based on the function code, and sends the response.
- Mutex Synchronization ensures thread-safe access to shared resources, while updates are communicated to the Modbus Data Manager.



Modbus Slave Task Overview Part III

- **Task Initialization:** Configures UART, initializes Modbus registers, enables RXNE interrupt, and signals readiness.
- **Notification Wait Loop:** Waits for UART IDLE line notifications, signaling a complete frame.
- **CRC Validation:** Checks frame integrity using CRC; discards invalid frames silently.
- **Request Processing:** Handles Modbus RTU requests, processes read/write operations and sends responses.
- **Synchronization:** Ensures thread-safe data processing using a mutex.
- **Error Handling:** Sends error messages to the Error Handler and handles failed feedback from the Modbus Data Manager.



Modbus Slave Task Files Preparation

Overview

- **Objective:** Prepare template .c/.h files for integrating/developing additional components, e.g., UART, Modbus Middleware, Modbus Synchronization Mutex.
- **Key Steps:**
 - Create `modbus_slave_task.h` with function code data structure, handle for the feedback queue and function prototypes.
 - Create `modbus_slave_task.c` Modbus Slave Task template to be updated as lessons progress, task notify function to be used via the UART2 IRQ Handler and Modbus Slave Tasks Start function.
- **Project Significance:** Sets up the Modbus Slave Task for further development.

 <code>modbus_slave_task</code>
<code>-TaskHandle_t modbus_slave_task_handle</code> <code>+QueueHandle_t modbus_feedback_queue_handle</code>
<code>-error_t handle_modbus_status_and_send_data_update(status, msg_type, address, quantity, *buffers)</code> <code>-void modbus_slave_task(*param)</code> <code>+BaseType_t modbus_slave_task_send_notification_from_isr(*xHigherPriorityTaskWoken)</code> <code>+void modbus_slave_tasks_start(void)</code>

UART Driver Overview & Integration

Overview

- **Objective:** Integrate and review the custom interrupt-driven UART driver using semaphore synchronization to efficiently handle communication within an RTOS environment and IDLE Line Detection for detecting end of Modbus Frames.
- **Key Steps:**
 - **Understand:** The structure and design of the custom UART driver.
 - **Integrate and Review:** Header file definitions, structures, static inline functions and public function prototypes.
 - **Integrate and Review:** Source file functions and interrupt handlers.
- **Project Significance:**
 - Gain an understanding of the UART driver initialization, synchronization, and communication with the Modbus Slave Task using task notifications.

Modbus Middleware Integration & Review

Overview

- **Objective:** Integrate the Modbus Middleware files and understand how to extend and customize them if required.
- **Key Steps:**
 - **Integration:** Integrate the files into the project, extend the error.h header file with additional definitions required by the new Modbus files.
 - **Review:** Understand the files, how they work, and which portions are extendable or customizable for application-specific use.
- **Project Significance:** Prepares the project for building the Modbus application on top of the underlying components.

Modbus Synchronization Mutex Implementation

Overview

- **Objective:** Introduce the use of the synchronization mutex and its purpose of ensuring data consistency between the Modbus Slave and Data Manager tasks and develop the header and source files.
- **Key Steps:**
 - **Intro and Header File:** Brief introduction and declare the mutex-related function prototypes in the header file.
 - **Source File:** Implement the mutex creation, lock, and unlock functionality.
- **Project Significance:**
 - Provides a simple and effective mechanism for ensuring data consistency between the Modbus Slave and Data Manager tasks.

Modbus Slave Task Update

Overview

- **Objective:** Expand the Modbus Slave Task initialize UART, lock/unlock the mutex, check the CRC from the received frame, extract the function code and integrate the Modbus Function Code processing functions into the task itself.
- **Key Steps:**
 - **Modbus Slave Task Update:** Develop the task further preparing it for basic tests with the Modbus Master Simulator.
- **Project Significance:**
 - Prepares the Modbus Slave Task for basic communication validation and seamless integration with other components.

UART2 IRQ Priority Update

Overview

- **Objective:** Define and set the IRQ priority for the UART2 ISR to support FreeRTOS functionality within the handler.
- **Key Steps:**
 - **Header File:** Update `irq.h` to include the UART2 IRQ priority definition.
 - **Source File:** Modify `irq.c` to configure the priority level in the NVIC, ensuring proper relative priority with other IRQs.
- **Project Significance:**
 - Ensures the **UART2 IRQ Handler** can call FreeRTOS functions (e.g., task notifications and giving the semaphore), allowing it to signal the Modbus Slave Task when a complete Modbus frame is received and to send responses back to the Modbus Master using `uart_transmit_bytes()`.

UART GPIO Definition & Initialization

Overview

- **Objective:** Review the Nucleo Board User Manual and STM32F446 datasheet to determine the GPIO pins and alternate function settings required for Modbus UART TX and RX communication.
- **Key Steps:**
 - **GPIO Definitions:** Define the UART TX and RX pins in `gpio_defs.h` based on the Nucleo Board User Manual and alternate function (AF) mapping.
 - **GPIO Initialization:** Add the TX and RX pin configurations to `gpio_init()` in `gpio.c` for proper UART functionality.
- **Project Significance:** Ensures UART communication for Modbus by enabling and configuring the necessary GPIO pins.

Modbus Slave Task Testing

Overview

- **Objective:** Conduct basic tests using the Modbus Master Simulator with Modbus registers initialized using test data.
- **Key Steps:**
 - **Startup Task Integration:** Integrate and call `modbus_slave_tasks_start()` from `main.c` to initialize the task.
 - **Modbus Master Simulator Tests:** Use Simply Modbus to send requests and validate the function codes and response handling.
- **Project Significance:**
 - Validates the communication and functionality of the Modbus Slave Task, ensuring readiness for further development.

The background features a complex network of thin, light-colored lines and small circles, forming various triangular shapes and a larger, interconnected web-like structure. The lines are primarily in shades of light green and yellow, while the circles are small and light grey. The overall effect is a subtle, technical, and geometric pattern.

Next: Modbus Slave Task Files Preparation
